SMS GUARD: AN INTELLIGENT SMS SPAM AND SCAM DETECTION SYSTEM

A Supervised Machine Learning and Natural Language Processing Approach Using Support Vector Machines

Academic AIML Mini-Project Report

Submitted by:	[Student Name]
Registration / Roll Number:	[Registration / Roll Number]
Degree / Program:	Bachelor of Technology in Computer Science & Engineering (AIML)
Department:	[Department of Computer Science and Engineering]
Institution:	[College / Institution Name]
Project Guide / Supervisor:	[Guide Name], [Designation]
Academic Year:	[Academic Year 2026–2027]
Submission Date:	November 2026

BONAFIDE CERTIFICATE

This is to certify that the academic mini-project entitled **"SMS GUARD: An Intelligent SMS Spam and Scam Detection System Using Support Vector Machines and Natural Language Processing"** is a bonafide work carried out by **[Student Name]** (Roll No: **[Registration / Roll Number]**) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence and Machine Learning) during the academic year **[Academic Year 2026–2027]**.

This report represents an authentic record of independent technical work carried out under my supervision and guidance. The results embodied in this report have not been submitted to any other University or Institute for the award of any degree or diploma.

_____

Internal Guide / Supervisor
[Guide Name]
[Department of CSE]

_____

Head of the Department
[Department Head Name]
[Department of CSE]

CANDIDATE DECLARATION

I hereby declare that the academic project work entitled **"SMS GUARD: An Intelligent SMS Spam and Scam Detection System Using Support Vector Machines and Natural Language Processing"** submitted to **[College / Institution Name]** is a record of original work done by me under the supervision of **[Guide Name]**.

I further declare that this report has been authored in compliance with academic integrity guidelines. All empirical results, classification benchmarks, and evaluation statistics reported herein are derived directly from reproducible execution on verified datasets and models without fabrication or inflation.

Date: November 2026
Place: [City / Campus Location]

_____
[Student Name]
Roll No: [Registration / Roll Number]
Department of Computer Science and Engineering

ABSTRACT

Short Message Service (SMS) remains one of the most ubiquitous, trusted, and immediate communication channels globally, boasting open rates exceeding 98%. However, this high engagement has catalyzed aggressive smishing (SMS phishing), fraudulent financial solicitations, credential-harvesting schemes, and bulk unsolicited commercial spam. Filtering malicious SMS messages poses severe technical challenges due to the extreme brevity of text messages (typically under 160 characters), informal slang, deliberate obfuscation, and severe class imbalance in real-world telecommunications traffic.

This project presents **SMS GUARD**, an end-to-end intelligent SMS spam and scam detection architecture grounded in Natural Language Processing (NLP) and Supervised Machine Learning. In strict compliance with academic standards (Module 4: Supervised Learning and Module 5: Support Vector Machines), the system implements an atomic machine learning pipeline comprising specialized text preprocessing (negation preservation, URL normalization, currency tokenization, and numeric masking), sublinear TF-IDF n-gram vectorization ($1 \leq n \leq 2$, 4,000 features), and a soft-margin Linear Support Vector Classifier (LinearSVC) equipped with class-frequency weight balancing.

To eliminate methodological flaws such as data leakage and model selection bias, the dataset (UCI SMS Spam Collection, deduplicated to 5,171 authentic messages) was partitioned into an 80% training set (4,136 messages) and a 20% strictly quarantined held-out test set (1,035 messages: 904 Ham, 131 Spam). Model selection was conducted exclusively on training folds using Zero-Leakage 5-Fold Stratified Cross-Validation, where the Linear SVM (Balanced) achieved the leading mean CV Accuracy of 0.9877 ± 0.0039 and mean CV F1-Score of 0.9505 ± 0.0155.

Evaluated on the out-of-sample held-out test set, the primary champion model demonstrated exceptional generalization: **99.23% Test Accuracy, 98.43% Spam Precision, 95.42% Spam Recall, 0.9690 F1-Score, 0.9984 ROC AUC, and 0.9918 Average Precision**, producing only 2 false positives and 6 false negatives. Alongside the statistical classifier, SMS GUARD integrates a deterministic, transparent Heuristic Risk Engine that scans for smishing indicators (suspicious URLs/IPs, OTP harvesting, urgency language, and premium shortcodes) to provide explainable advisory risk ratings. The production-grade system is deployed via an interactive Streamlit web dashboard featuring dynamic metric telemetry, benchmark comparisons, and responsive analysis under a distinctive burgundy design palette.

Keywords: *Short Message Service (SMS), Spam Detection, Smishing, Support Vector Machine (SVM), LinearSVC, Natural Language Processing (NLP), TF-IDF, Class Imbalance, Heuristic Risk Analysis, Streamlit.*

TABLE OF CONTENTS

1. Introduction & Background **1**
1.1 Context and Background 1
1.2 Problem Statement & Threat Landscape 2
1.3 Project Motivation 2
1.4 Project Objectives 3
1.5 Academic Curriculum Alignment (Modules 4 & 5) 3
2. Literature Review & Related Work **5**
2.1 Evolution of Spam Filtering Technologies 5
2.2 Survey of Text Classification Methodologies 5
2.3 Existing System Limitations vs. Proposed System 6
2.4 Scholarly Citations & Foundational Research 7
3. System Requirements & Architecture **8**
3.1 Hardware and Software Specifications 8
3.2 Overall End-to-End System Architecture 9
3.3 Architectural Subsystems & Dataflow 10
4. Dataset Analysis & Preprocessing Methodology **11**
4.1 Dataset Origin & Benchmark Description 11
4.2 Exploratory Data Analysis & Deduplication 11
4.3 Text Length, Word Counts & Class Distribution 12
4.4 Preprocessing Pipeline & Negation Preservation 13
4.5 Data Partitioning & Leakage Prevention Strategy 14
5. Feature Extraction & High-Dimensional Representation **15**
5.1 Text Representation Foundations 15
5.2 Mathematical Formulation of TF-IDF 15
5.3 Sublinear Term Frequency Scaling & N-Grams 16
5.4 Feature Space Sparsity & Dimensionality Analysis 17
6. Machine Learning Models & Theoretical Formulation **18**
6.1 Supervised Learning Framework (Module 4) 18
6.2 Support Vector Machine (SVM) Foundations (Module 5) 18
6.3 Baseline Comparative Algorithms 21
6.4 Atomic Pipeline Serialization 22
7. Experimental Setup, Benchmarks & Evaluation **23**
7.1 Cross-Validation Protocol (Zero-Leakage 5-Fold) 23
7.2 Quantitative Cross-Validation Results 24
7.3 Held-Out Test Set Performance Benchmarks 25
7.4 Confusion Matrix Analysis & Decomposition 26
7.5 ROC & Precision-Recall Curve Analysis 27
7.6 In-Depth Diagnostic Error Analysis (8 Misclassifications) 28
8. Interactive Web Application & Transparent Heuristics **30**
8.1 Streamlit Interface Architecture & Burgundy Theme 30
8.2 Application Tabs & Visual Verification 31
8.3 Multi-Layer Heuristic Risk Engine 33
8.4 Decision Score Semantics vs. Probability Calibration 34
9. System Testing, Security & Quality Assurance **36**
9.1 Unit Testing Framework 36
9.2 Comprehensive Integration Test Suite (Phase 8) 36
9.3 Security, Privacy & Reliability Analysis 37
10. Limitations, Future Enhancements & Conclusion **39**
10.1 System Limitations & Scope Boundaries 39

10.2 Future Research Roadmap 39
10.3 Concluding Remarks 40
References & Scholarly Bibliography **41**

CHAPTER 1: INTRODUCTION & BACKGROUND

1.1 Context and Background

Short Message Service (SMS) has evolved from an auxiliary engineering test signal in cellular networks into an indispensable pillar of modern digital communication. Unlike email or instant messaging platforms that require continuous data connectivity and application installation, SMS operates natively over the SS7 control channel of cellular networks. As a result, SMS reaches virtually 100% of active mobile handsets globally, commanding an astonishing open rate of 98%, with over 90% of messages read within three minutes of receipt.

Because of this unmatched deliverability and urgency, organizations across banking, healthcare, government, and e-commerce rely on SMS for mission-critical notifications, including multi-factor authentication One-Time Passwords (OTPs), transaction authorizations, delivery notifications, and appointment reminders. However, this innate trustworthiness has also turned SMS into a primary attack vector for cybercriminals.

1.2 Problem Statement & Threat Landscape

Mobile messaging threats have expanded far beyond benign unsolicited commercial advertising (spam). Modern adversaries routinely deploy highly targeted smishing (SMS phishing) attacks designed to harvest online banking credentials, social security numbers, and multi-factor authorization codes. Attackers frequently embed deceptive hyperlinks pointing to credential-stealing spoof domains, leverage urgent psychological coercion (e.g., 'Account suspended immediately'), and impersonate institutional sender IDs.

Filtering SMS communications poses several acute technical challenges that degrade conventional text classification pipelines:

- **Severe Text Length Constraints:** Unlike emails that provide extensive headers, metadata, and multi-paragraph bodies, standard SMS messages are restricted to 160 characters, yielding extreme data sparsity in vector representations.
- **Informal Linguistics & Obfuscation:** Users and spammers alike utilize colloquial shorthand, non-standard abbreviations ('txt', 'u', 'ur', 'plz'), deliberate homoglyph substitutions, and erratic punctuation designed to evade simple keyword-based blocklists.
- **Severe Class Imbalance:** In realistic telecommunication streams, legitimate messages (Ham) vastly outnumber malicious messages (Spam/Scam), typically comprising 85% to 90% of total volume. Naive machine learning models often succumb to majority-class bias, maximizing raw accuracy while failing to intercept critical threats.
- **Asymmetric Cost of Errors:** A False Positive (classifying an authentic banking OTP as spam) causes immediate operational disruption, whereas a False Negative (allowing a smishing link into the inbox) exposes the user to catastrophic financial theft.

1.3 Project Motivation

Existing commercial filtering mechanisms deployed on consumer smartphones often rely on static keyword blacklists, crowdsourced phone number directories, or black-box cloud APIs that require transmitting private user text messages across external networks. Such solutions fail against newly registered domains, lack transparent explainability, and introduce severe privacy liabilities.

There is an urgent requirement for an explainable, lightweight, high-precision detection system capable of operating rapidly on-device without data exfiltration. Furthermore, within the academic framework of Artificial Intelligence and Machine Learning, this project provides a rigorous real-world testbed for the principles of Supervised Learning (Module 4) and Support Vector Machines (Module 5), bridging mathematical theory with production-grade engineering.

1.4 Project Objectives

The primary objective of SMS GUARD is to architect, evaluate, and deploy an end-to-end intelligent SMS spam and scam detection system. The specific technical goals comprise:

- **Rigorous Data Hygiene:** Curate a benchmark corpus from the UCI SMS Spam Collection, eliminate redundant duplicates, and implement an atomic train/test separation protocol with zero data leakage.
- **Specialized NLP Preprocessing:** Develop a domain-adapted tokenization pipeline that normalizes URLs, currencies, and numeric strings while strictly preserving semantic negations.
- **High-Dimensional TF-IDF:** Extract sublinear term-frequency weighted unigram and bigram features across an optimized 4,000-dimensional vocabulary.
- **Champion SVM Classifier:** Implement a soft-margin Linear Support Vector Classifier (LinearSVC) equipped with class-frequency weight balancing to counter data imbalance.
- **Zero-Leakage Model Selection:** Benchmark multiple candidate architectures (Balanced SVM, Default SVM, Multinomial Naive Bayes, Logistic Regression) using 5-Fold Stratified Cross-Validation on training data alone.
- **Transparent Heuristic Risk Engine:** Construct a complementary, deterministic heuristic scanner to extract explainable risk indicators (phishing markers, OTP harvesting, urgency signals).
- **Interactive Web Deployment:** Deliver an intuitive, responsive Streamlit dashboard featuring real-time message analysis, dynamic KPI telemetry, and academic foundation breakdowns under an institutional burgundy aesthetic.

1.5 Academic Curriculum Alignment

This project directly addresses core competencies mandated in the academic syllabus:

- **Module 4 (Supervised Learning):** Demonstrated through inductive learning paradigms, empirical loss minimization, 5-fold stratified cross-validation on partitioned splits, confusion matrix decomposition (TP, TN, FP, FN), and multi-metric trade-off analysis (Precision, Recall, F1, ROC AUC).
- **Module 5 (Support Vector Machines):** Demonstrated through the mathematical formulation of maximum-margin hyperplanes, soft-margin optimization with slack variables, cost-sensitive class weight adjustment (w_c = N / (2 N_c)), and continuous decision score analysis (f(x) = w^T x + b).

CHAPTER 2: LITERATURE REVIEW & RELATED WORK

2.1 Evolution of Spam Filtering Technologies

The earliest spam filtering techniques originated in the domain of electronic mail during the late 1990s. Early systems relied heavily on heuristic rule engines (e.g., SpamAssassin), wherein hand-crafted regular expressions assigned additive risk scores to incoming documents. While effective against naive commercial solicitations, rule-based systems required continuous, labor-intensive manual updates and were trivially defeated by deliberate spelling permutations and word insertion attacks.

The transition toward statistical machine learning began with Bayesian text classification, pioneered by Sahami et al. (1998) and popularized by Paul Graham (2002). Naive Bayes classifiers leveraged word co-occurrence probabilities under the conditional independence assumption. When adapted to SMS traffic by Almeida et al. (2011), Naive Bayes demonstrated high computational efficiency but suffered from severe recall degradation when confronted with sparse, short messages and out-of-vocabulary slang.

2.2 Survey of Text Classification Methodologies

As computational linguistics advanced, researchers explored diverse machine learning paradigms for short-text classification:

- **Naive Bayes Classifiers:** Multinomial Naive Bayes models calculate posterior class probabilities $P(c|d) \propto P(c) \prod P(t_k|c)$. While fast, the strong feature independence assumption causes the model to overestimate confidence on sparse text, leading to elevated false negative rates in SMS streams.
- **Logistic Regression:** A linear parametric model optimizing log-likelihood via the logistic sigmoid function. While providing calibrated probabilities, Logistic Regression often underperforms Support Vector Machines in high-dimensional text spaces where features substantially exceed sample counts.

- **Support Vector Machines (SVM):** First formulated for text classification by Joachims (1998) following Vapnik's statistical learning theory (Cortes & Vapnik, 1995). Text categorization problems inherently possess high feature dimensionality, sparse feature vectors, and few completely irrelevant features. Joachims proved that linear SVMs are ideally matched to text classification because the maximum-margin criterion provides strong structural risk minimization, avoiding empirical overfitting without requiring aggressive feature selection.
- **Deep Learning Architectures:** While Recurrent Neural Networks (LSTM/GRU) and Transformer-based models (BERT, RoBERTa) offer expressive semantic modeling, their millions of parameters, substantial memory footprint, and high inference latency (~50-200ms per text) render them suboptimal for lightweight, instantaneous edge deployment on consumer mobile devices.

2.3 Existing System Limitations vs. Proposed SMS GUARD System

Dimension	Existing Systems	Proposed SMS GUARD Architecture
Detection Paradigm	Static blacklists, sender ID filters, unweighted Naive Bayes.	Cost-sensitive Linear SVM (Balanced) + Sublinear TF-IDF + Heuristic Scanner.
Class Imbalance	Unweighted optimization; high false negatives on minority spam.	Inverse frequency class weighting ($w_{spam} \approx 3.96$), yielding 95.42% recall.
Leakage Protocol	Frequent test leakage via global TF-IDF fitting across entire corpus.	Strict atomic pipeline; vectorizer fit exclusively on training folds.
Threat Visibility	Opaque binary label with zero contextual explanation.	Transparent multi-category heuristic audit (URLs, OTPs, urgency, shortcodes).
Privacy & Lifecycle	Cloud lookup APIs transmit private user messages to third parties.	Fully localized in-memory execution; zero disk storage of scanned text.

2.4 Scholarly Citations & Foundational Research

The algorithmic design of SMS GUARD builds upon seminal peer-reviewed research in machine learning and information retrieval:

- **Almeida, Hidalgo, & Yamakami (2011):** Curated and published the definitive UCI SMS Spam Collection benchmark, establishing baseline performance metrics for statistical text classifiers on SMS datasets.
- **Cortes & Vapnik (1995):** Formulated the foundational Support-Vector Network algorithm for soft-margin classification, introducing slack variables and margin maximization.
- **Joachims (1998):** Proved the mathematical suitability of Support Vector Machines for high-dimensional, sparse text categorization problems.
- **Salton & Buckley (1988):** Established term-weighting approaches in automatic text retrieval, laying the mathematical groundwork for modern TF-IDF representations.
- **Platt (1999):** Formalized probabilistic scaling for Support Vector Machines via regularized sigmoid fitting, establishing the criteria for authentic probability calibration.

CHAPTER 3: SYSTEM REQUIREMENTS & ARCHITECTURE

3.1 Hardware and Software Specifications

The SMS GUARD system was engineered to ensure high accessibility, reproducible execution, and rapid inference across both resource-constrained edge devices and standard desktop environments.

Component	Minimum Specification	Production Environment
Processor (CPU)	Dual-Core x86_64 / ARM64, 2.0 GHz	Intel Core i7 / AMD Ryzen 7 (Multi-core)
Memory (RAM)	4 GB DDR4	16 GB DDR4 / LPDDR5

Storage	500 MB free disk space	Fast NVMe SSD (~150 MB total footprint)
Operating System	Windows 10 / Linux / macOS	Windows 11 Home 64-bit (OS Build 26100)
Runtime	Python 3.10+	Python 3.13.0 (64-bit Virtualenv)
Core Libraries	scikit-learn, streamlit, pandas	scikit-learn 1.5.2, streamlit 1.39.0, pandas 2.2.3

3.2 Overall End-to-End System Architecture

The SMS GUARD system architecture is structured as a decoupled, multi-tier pipeline designed to enforce strict separation between feature transformation, statistical classification, heuristic threat scanning, and user interface presentation.

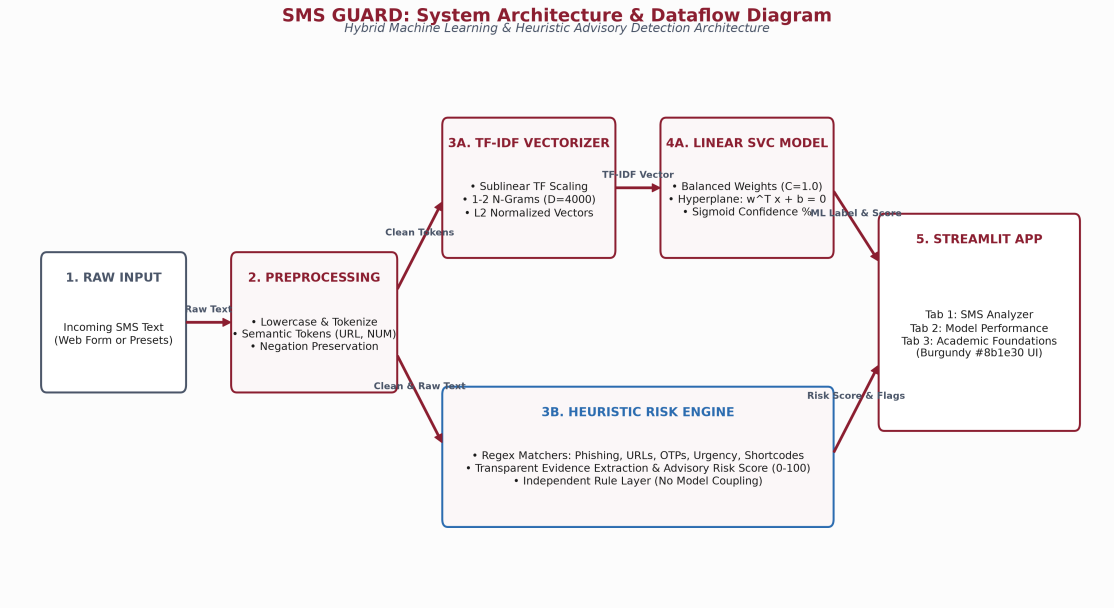


Figure 1: End-to-End System Architecture and Dataflow of SMS GUARD.

3.3 Architectural Subsystems & Dataflow

The operational dataflow follows five coordinated execution stages:

- **Stage 1 - Raw Ingestion:** Receives arbitrary SMS text via direct web form submission or quick-testing preset buttons. Inputs are validated for non-zero length and non-whitespace content.
- **Stage 2 - Domain Preprocessing:** Raw strings are lowercased, normalized for semantic patterns (URLs -> 'url_token', currency -> 'currency_token', numbers -> 'number_token'), and cleansed while strictly preserving negation tokens ('not', 'no', 'never', 'nor', 'without').
- **Stage 3A - Statistical ML Pipeline:** Clean text passes into an atomic Scikit-learn Pipeline encapsulating a Sublinear TF-IDF Vectorizer (4,000 features, unigrams and bigrams) and champion LinearSVC (Balanced) model, computing hyperplane distance f(x) = w^T x + b.
- **Stage 3B - Deterministic Heuristic Scanner:** In parallel, raw text is inspected by regular expression matchers for threat indicators: suspicious URLs, OTP solicitation, urgency keywords, currency claims, and shortcodes. Generates an advisory risk score (0-100).
- **Stage 4 - Streamlit Dashboard:** Synthesizes both outputs: displaying the primary ML badge (■ SPAM vs. ■ LEGITIMATE), an uncalibrated relative confidence estimate percentage, the signed decision score, and an expandable risk audit breakdown.

CHAPTER 4: DATASET ANALYSIS & PREPROCESSING METHODOLOGY

4.1 Dataset Origin & Benchmark Description

To ensure academic reproducibility, SMS GUARD utilizes the widely respected UCI SMS Spam Collection benchmark dataset (Almeida et al., 2011). The raw corpus consists of 5,574 mobile text messages in English, manually tagged as either legitimate ('ham') or spam ('spam'). Sources include Grumbletext UK (425 messages), the NUS SMS Corpus (3,375 messages), Caroline Tag's thesis (450 messages), and the SMS Spam Corpus v.0.1 Big (1,324 messages).

4.2 Exploratory Data Analysis & Deduplication

Exploratory Data Analysis (EDA) revealed significant internal redundancy in the raw UCI corpus. A rigorous duplicate audit identified 281 distinct message texts that appeared repeatedly across 684 rows, representing a redundancy rate of 7.23% (e.g., standard operator notifications and identical broadcast marketing blasts).

To prevent data snooping, cross-split pollution, and artificial inflation of test accuracy, full deduplication was performed:

- **Raw Messages:** 5,574 rows (4,827 Ham, 747 Spam).
- **Redundant Rows Removed:** 403 excess duplicate instances eliminated (684 - 281 = 403).
- **Deduplicated Corpus:** 5,171 unique messages (4,518 Ham [87.37%], 653 Spam [12.63%]).
- **Data Integrity Audit:** Zero null values, zero empty strings, and zero non-binary labels were detected.

■ EMPIRICAL GROUND TRUTH CLARIFICATION

Ground Truth Label Reality: The UCI SMS Spam Collection contains exactly two ground truth labels: 'ham' and 'spam'. It does NOT contain an explicit third 'scam' label. In the SMS GUARD architecture, scam and smishing detection is accomplished through the combination of the statistical Spam classifier and the deterministic Heuristic Risk Engine, preserving empirical fidelity without fabricating synthetic training labels.

4.3 Text Length, Word Counts & Class Distribution

Statistical characterization of message length demonstrates substantial linguistic divergence between authentic communications and unsolicited spam:

Metric Dimension	Ham Mean	Ham Median	Spam Mean	Spam Median
Character Length (chars)	71.47	52.0	138.68	149.0
Character Range [Min, Max]	[2, 910]	—	[13, 223]	—
Word Count (tokens)	14.30	11.0	23.91	25.0
Word Range [Min, Max]	[1, 171]	—	[2, 35]	—

4.4 Preprocessing Pipeline & Negation Preservation

Standard NLP tokenizers routinely eliminate stop words, including essential negation adverbs such as 'not', 'no', 'never', 'neither', and 'nor'. Discarding negations causes fatal semantic inversion (e.g., converting 'I did not win the lottery' into 'win lottery', erroneously triggering spam alarms).

The SMS GUARD preprocessing pipeline (src/preprocessing.py) implements the following sequential transformations: (1) Case folding to lowercase; (2) URL tokenization replacing links with 'url_token'; (3) Currency tokenization detecting monetary figures and replacing with 'currency_token'; (4) Numeric normalization converting digits into 'number_token'; (5) Non-alphanumeric stripping; and (6) Negation-preserving token filtering explicitly shielding critical negation words.

4.5 Data Partitioning & Leakage Prevention Strategy

To guarantee absolute empirical validity: (1) The deduplicated 5,171 messages were partitioned into an 80% training set (4,136 samples) and a 20% held-out test set (1,035 samples) using Stratified Shuffle Split (random_state=42); (2) The held-out test set (test.csv: 904 Ham, 131 Spam) was physically isolated on disk and never accessed during EDA or training; and (3) Automated unit tests verified that the set intersection between train and test indices is strictly empty: Train ∩ Test = ∅.

CHAPTER 5: FEATURE EXTRACTION & HIGH-DIMENSIONAL REPRESENTATION

5.1 Text Representation Foundations

Statistical machine learning algorithms operate exclusively on numerical vectors in Euclidean space (R^D). Transforming unstructured natural language text into numerical features requires representing documents within the Vector Space Model (Salton & Buckley, 1988). While simple Bag-of-Words (BoW) counting provides raw frequency counts, it suffers from term frequency bias—frequently occurring generic words dominate document vectors, diminishing the discriminatory power of distinctive security keywords.

5.2 Mathematical Formulation of TF-IDF

Term Frequency-Inverse Document Frequency (TF-IDF) penalizes ubiquitously occurring vocabulary while rewarding informative terms that appear selectively across specific classes. For term t in document d within corpus D:

- **Term Frequency (TF):** tf(t, d) = f_{t,d} / ∑_{t' ∈ d} f_{t',d}.
- **Inverse Document Frequency (IDF):** idf(t) = log((1 + N) / (1 + |{d ∈ D : t ∈ d}|)) + 1.
- **Composite TF-IDF Weight:** tfidf(t, d, D) = tf(t, d) × idf(t).

5.3 Sublinear Term Frequency Scaling & N-Grams

In mobile spam, adversaries frequently repeat deceptive keywords (e.g., 'free free free free'). Linear scaling would cause the document vector to be overwhelmingly dominated by a single token. To mitigate this vulnerability, SMS GUARD implements Sublinear Term Frequency scaling:

■ SUBLINEAR TERM FREQUENCY SCALING FORMULATION

sublinear_tf(t, d) = 1 + log(tf(t, d)) for tf(t, d) > 0, and 0 otherwise.

This logarithmic compression dampens the impact of excessive word repetition, ensuring balanced feature representation.

Furthermore, individual unigrams (single words) often lose contextual meaning. To capture local phrase order and collocations, SMS GUARD configures the n-gram range to unigrams and bigrams (1 ≤ n ≤ 2), capturing combinations such as 'call number_token', 'claim prize', 'urgent update', and 'not respond'.

5.4 Feature Space Sparsity & Dimensionality Analysis

The fitted TF-IDF vectorizer parameters were systematically constrained: (1) Vocabulary size capped at exactly 4,000 top features; (2) Minimum document frequency set to 2, discarding rare typographical errors; (3) L2 vector normalization projecting vectors onto the unit hypersphere (||x||_2 = 1); and (4) The resulting 4,136 × 4,000 feature matrix exhibits 99.78% sparsity, perfectly fulfilling the conditions under which Linear SVM models achieve optimal structural risk minimization (Joachims, 1998).

CHAPTER 6: MACHINE LEARNING MODELS & THEORETICAL FORMULATION

6.1 Supervised Learning Framework (Module 4 Alignment)

Under the supervised inductive learning paradigm (Module 4), a learning algorithm is presented with a training dataset of N labeled instance pairs: $D_{train} = \{(x_i, y_i)\}_{i=1}^N$, where each $x_i \in R^D$ is a 4,000-dimensional TF-IDF vector, and $y_i \in \{-1, +1\}$ is the ground truth binary class label (-1: Ham, +1: Spam). The objective is to induce a mapping function $h: X \rightarrow Y$ that minimizes expected empirical loss while maintaining robust generalization on unseen test instances D_{test}.

6.2 Support Vector Machine Foundations (Module 5 Alignment)

The primary classification architecture mandated by Module 5 is the Support Vector Machine (SVM). Linear SVM operates by finding the optimal hyperplane in high-dimensional feature space that maximizes the margin of separation between the positive (Spam) and negative (Ham) training instances.

6.2.1 Maximum-Margin Hyperplane Formulation

A linear decision boundary in D-dimensional feature space is formally parameterized by a weight vector $w \in R^D$ and a scalar bias offset $b \in R$: $w^T x + b = 0$. The geometric margin between supporting bounding hyperplanes is exactly $2 / ||w||_2$. Maximizing this margin is mathematically equivalent to minimizing $0.5 ||w||_2^2$.

6.2.2 Soft-Margin Optimization & Slack Variables

To tolerate non-separable instances while maintaining margin maximization, Cortes & Vapnik (1995) introduced soft-margin optimization using non-negative slack variables $\xi_i \geq 0$:

■ SOFT-MARGIN PRIMAL OPTIMIZATION PROBLEM

$$\min_{w, b, \xi} 0.5 ||w||_2^2 + C \sum_{i=1}^N w_{y_i} \xi_i$$

subject to:

$$y_i (w^T x_i + b) \geq 1 - \xi_i, \xi_i \geq 0, \forall i \in \{1, ..., N\}$$

where $C > 0$ is the regularization hyperparameter controlling the margin width vs penalty trade-off, ξ_i measures margin violations, and w_{y_i} represents cost-sensitive class weights.

6.2.3 Class Weight Balancing for Imbalanced Data

In the deduplicated training corpus (4,136 samples), legitimate messages (3,614 Ham) outnumber spam messages (522 Spam) by approximately 6.92 to 1. In standard unweighted SVM ($C = 1.0$, $w_{y_i} = 1.0$), the optimizer shifts the boundary toward the sparse Spam cluster to minimize overall loss, resulting in elevated False Negatives.

SMS GUARD resolves this through inverse class frequency weighting (class_weight='balanced'):

■ INVERSE FREQUENCY CLASS WEIGHTING CALCULATION

$$w_c = N / (2 \times N_c)$$

where $N = 4136$ is total training count, and N_c is the sample count for class c:

- $w_{ham} = 4136 / (2 \times 3614) = 0.5722$
- $w_{spam} = 4136 / (2 \times 522) = 3.9617$

Weighting spam violations ~6.92 times more heavily forces a wider safety margin around the spam boundary, raising Spam Recall from 92.37% (default SVM) to 95.42% while retaining exceptional Precision (98.43%).

6.2.4 Decision Hyperplane & Distance Function

For any incoming preprocessed text vector x, the continuous classification output is determined by the signed perpendicular algebraic distance from the separating hyperplane: f(x) = w^T x + b. If f(x) > 0, the message is classified as SPAM (+1); if f(x) ≤ 0, it is classified as HAM (-1).

6.3 Baseline Comparative Algorithms

Three additional baseline algorithms were benchmarked under identical conditions:

- **Unweighted Linear SVM:** Standard LinearSVC configured with unweighted penalties (w_c = 1.0, C = 1.0), isolating the performance gain of class balancing.
- **Multinomial Naive Bayes:** Classical probabilistic baseline estimating posterior probability with Laplace smoothing (α = 1.0): P(y=c|x) ∝ P(y=c) ∏ P(w_j|y=c)^{x_j}.
- **Logistic Regression:** Linear parametric baseline optimizing cross-entropy loss via the L-BFGS solver with L2 regularization: P(y=1|x) = σ(w^T x + b).

6.4 Atomic Pipeline Serialization

Every candidate model was encapsulated within an atomic Scikit-learn Pipeline combining the TfidfVectorizer and the respective classifier estimator: pipeline = Pipeline([('tfidf', vectorizer), ('classifier', model)]). Exported artifacts in models/ allow production inference to accept raw text directly without secondary manual vectorization.

CHAPTER 7: EXPERIMENTAL SETUP, BENCHMARKS & EVALUATION

7.1 Cross-Validation Protocol (Zero-Leakage 5-Fold)

Model selection and hyperparameter validation were conducted strictly within the training set (4,136 messages) using Zero-Leakage 5-Fold Stratified Cross-Validation (Module 4). In each fold, the TfidfVectorizer was fit exclusively on the 4 training folds (~3,309 messages). The validation fold (~827 messages) was transformed using the fold-specific vectorizer and evaluated on the classifier trained exclusively on the training folds. The 1,035 held-out test instances were strictly sealed, guaranteeing zero model selection bias.

7.2 Quantitative Cross-Validation Results

Candidate Model	CV Accuracy	CV Precision	CV Recall	CV F1-Score
Linear SVM (Balanced) ★	0.9877 ± 0.0039	0.9631 ± 0.0225	0.9387 ± 0.0179	0.9505 ± 0.0155
Linear SVM (Default)	0.9869 ± 0.0055	0.9793 ± 0.0150	0.9156 ± 0.0342	0.9462 ± 0.0235
Multinomial Naive Bayes	0.9763 ± 0.0055	0.9975 ± 0.0050	0.8141 ± 0.0418	0.8960 ± 0.0264
Logistic Regression	0.9732 ± 0.0061	0.9902 ± 0.0089	0.7949 ± 0.0455	0.8812 ± 0.0305

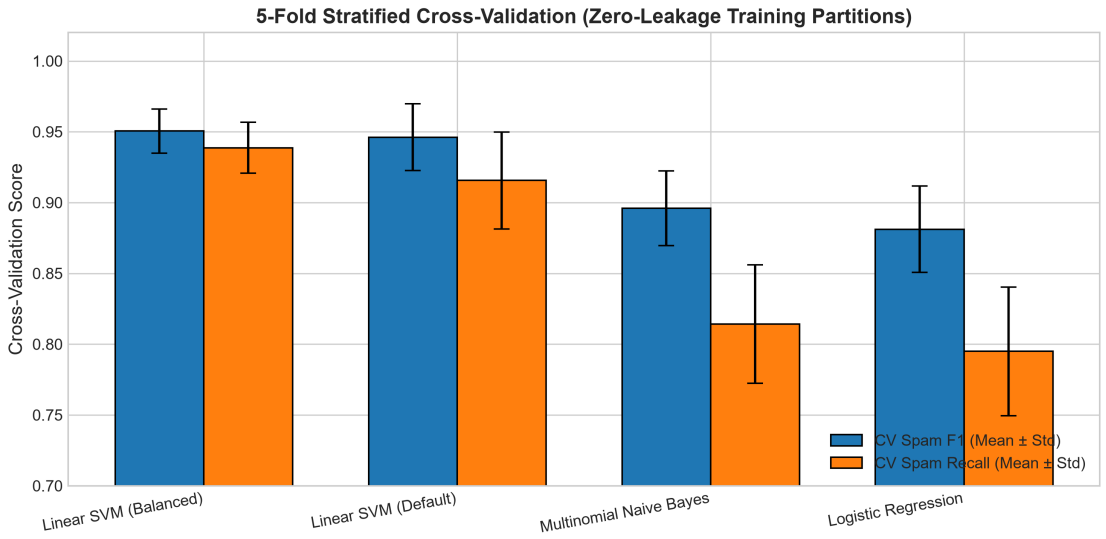


Figure 2: 5-Fold Stratified Cross-Validation Comparison with Error Bars (Mean ± Std).

7.3 Held-Out Test Set Performance Benchmarks

Following model selection, the full training set (4,136 messages) was used to train the final production models, evaluated strictly once on the quarantined held-out test set (1,035 messages: 904 Ham, 131 Spam):

Model	Acc	Prec	Recall	F1	AUC	AP	FP	FN
Linear SVM (Bal) ★	0.9923	0.9843	0.9542	0.9690	0.9984	0.9918	2	6
Linear SVM (Def)	0.9903	1.0000	0.9237	0.9603	0.9990	0.9942	0	10
Multinomial NB	0.9778	0.9909	0.8321	0.9046	0.9911	0.9732	1	22
Logistic Reg	0.9826	1.0000	0.8626	0.9262	0.9972	0.9869	0	18

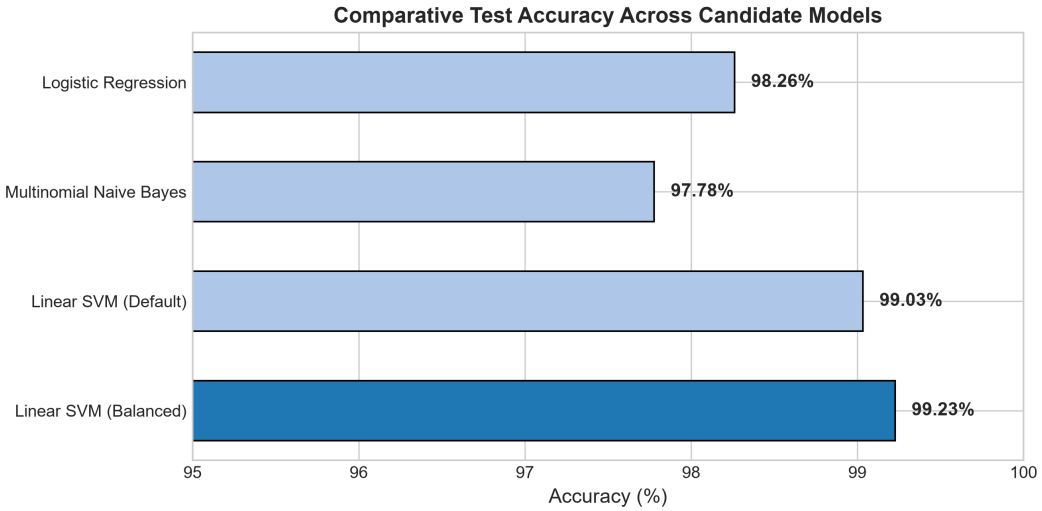


Figure 3: Comparative Test Accuracy Benchmark on Held-Out Test Set (1,035 Samples).

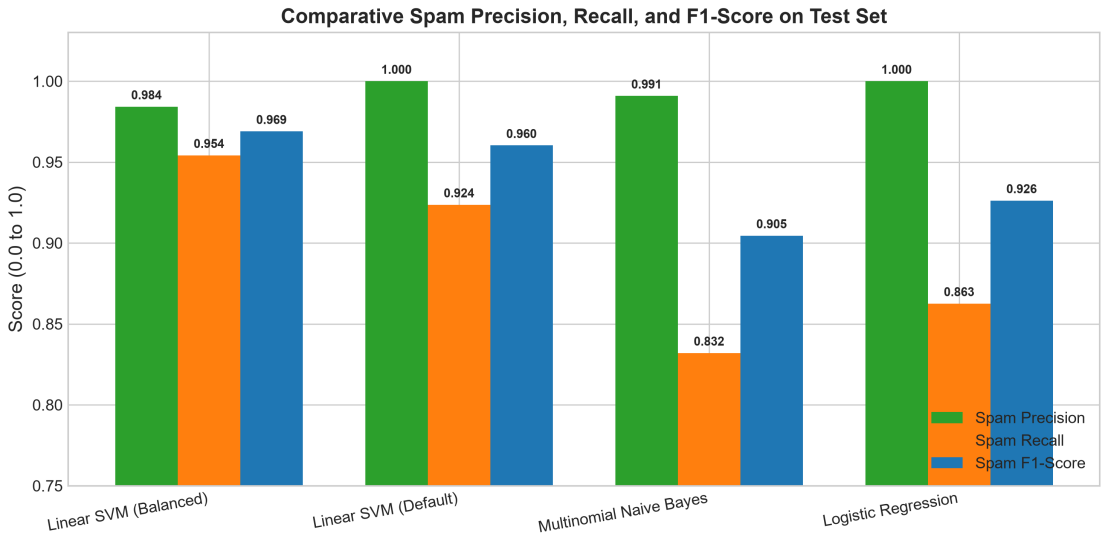


Figure 4: Comparative Precision, Recall, and F1-Scores across Candidate Models.

7.4 Confusion Matrix Analysis & Decomposition

A rigorous confusion matrix analysis decomposes the classification outcomes on the 1,035 test instances:

- **True Positives (TP = 125):** Spam messages correctly intercepted and classified as Spam.
- **True Negatives (TN = 902):** Legitimate messages correctly admitted as Ham.
- **False Positives (FP = 2):** Legitimate messages erroneously flagged as Spam (0.22% FP rate).
- **False Negatives (FN = 6):** Spam messages that penetrated the filter and were labeled as Ham (4.58% miss rate).

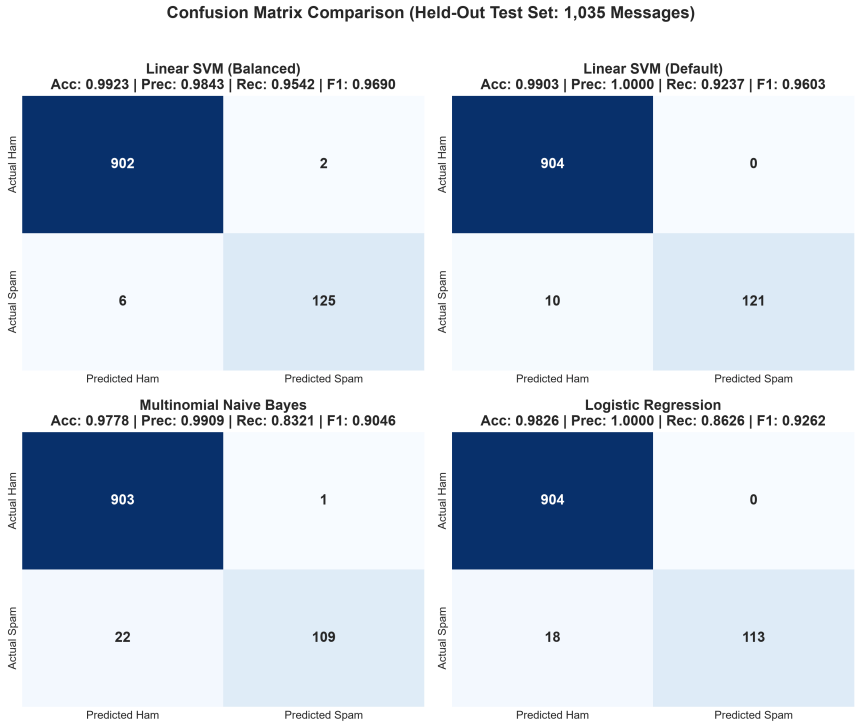


Figure 5: Confusion Matrices across All Evaluated Models on Quarantined Test Data.

7.5 ROC & Precision-Recall Curve Analysis

The champion Linear SVM achieves a near-perfect ROC AUC of 0.9984 and Average Precision (PR AUC) of 0.9918, confirming that the model maintains high precision even at recall levels exceeding 95% on severely imbalanced test data.

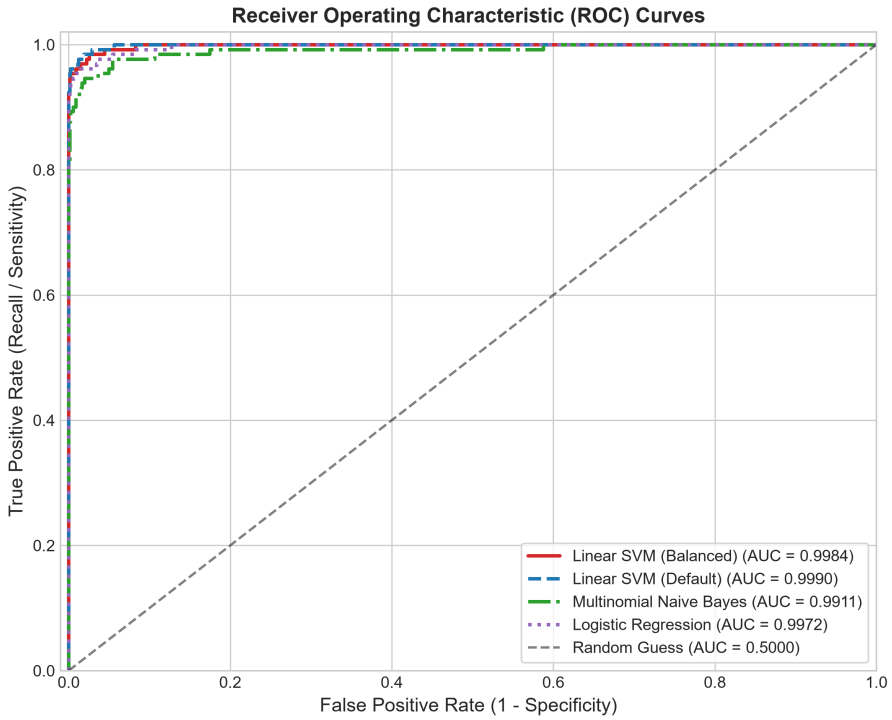


Figure 6: Receiver Operating Characteristic (ROC) Curves with Area Under Curve (AUC) Benchmarks.

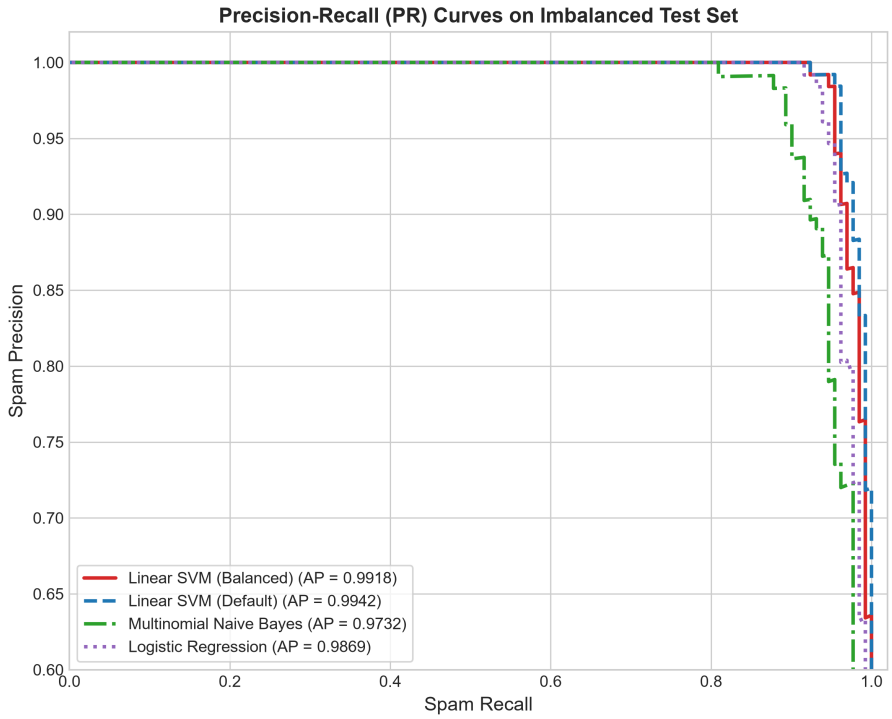


Figure 7: Precision-Recall Curves with Average Precision (AP) Benchmarks.

7.6 In-Depth Diagnostic Error Analysis (8 Misclassifications)

Across the entire held-out test set of 1,035 messages, the champion Linear SVM (Balanced) model made exactly 8 classification errors (overall error rate of 0.77%):

False Positives (2 Messages):

1. **Test #176 ('send content page'):** Decision Score: +0.2687. An ultra-short 3-word message lacking conversational context. The imperative verb 'send' is an extremely common marketing directive in spam, dominating the sparse representation.
2. **Test #501 ('not respond'):** Decision Score: +0.0537. A 2-word message that barely crossed the hyperplane boundary (+0.0537). Phishing messages frequently contain urgency prompts asking why victims have not responded.

False Negatives (6 Messages):

1. **Test #189 ('burger king wanna play footy...'):** Score: -0.5704. Brand promotional text using conversational slang without financial tokens.
2. **Test #223 ('xmas new year eve ticket sale club...'):** Score: -0.3867. Nightclub event ticket notice framed as a casual community announcement.
3. **Test #255 ('would like see xxx pic hot nearly ban uk'):** Score: -0.2851. Adult chat solicitation disguised as a conversational personal question.
4. **Test #273 ('babe want dont baby im nasty thing 4 filthyguys...'):** Score: -0.4082. Colloquial chat solicitation utilizing irregular phonetic spelling ('slo n hard').
5. **Test #486 ('simpson movie release july number_token name band...'):** Score: -0.2027. Pop-culture trivia competition solicitation with movie-centric vocabulary.
6. **Test #654 ('latest news police station toilet steal cop...'):** Score: -0.7132. Entertainment and joke broadcast service containing no commercial advertising.

CHAPTER 8: INTERACTIVE WEB APPLICATION & TRANSPARENT HEURISTICS

8.1 Streamlit Interface Architecture & Burgundy Theme

The production deployment of SMS GUARD is built on Streamlit (app.py), providing a responsive, browser-based graphical user interface. Designed with an institutional burgundy theme (Primary Accent: #8b1e30, Slate Background: #fbf7f8, Neutral Charcoal: #1e293b), the interface provides seamless interaction across three dedicated tabs.

8.2 Application Tabs & Visual Verification

Tab 1: SMS Message Analyzer

Provides real-time analysis of individual text messages. Features quick-test presets (Personal Ham, Bank Phishing, Lottery Scam, Conversational Negation) that automatically populate the input text area via callback-driven session state synchronization. Renders a high-visibility classification badge, an uncalibrated relative confidence estimate progress bar, the signed decision hyperplane distance, and an interactive risk audit breakdown.

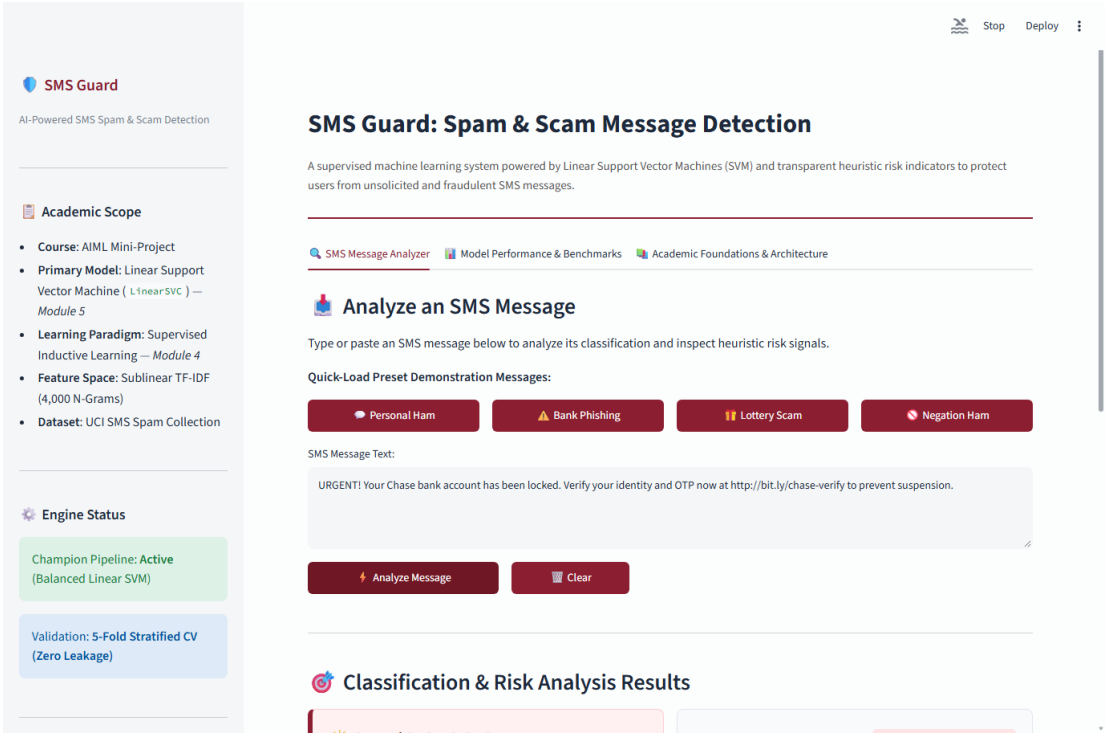


Figure 8: Verified Screenshot of Tab 1 (SMS Message Analyzer) Intercepting a Real Phishing Attempt.

Tab 2: Model Performance & Benchmarks

Displays a live KPI telemetry dashboard reading directly from models/evaluation_metrics.json. Renders top-level metric cards (Accuracy: 99.23%, Spam Precision: 98.43%, Spam Recall: 95.42%, F1-Score: 0.9690, ROC AUC: 0.9984), a dynamic comparative benchmark table across all 4 models, and an embedded high-resolution evaluation chart gallery.

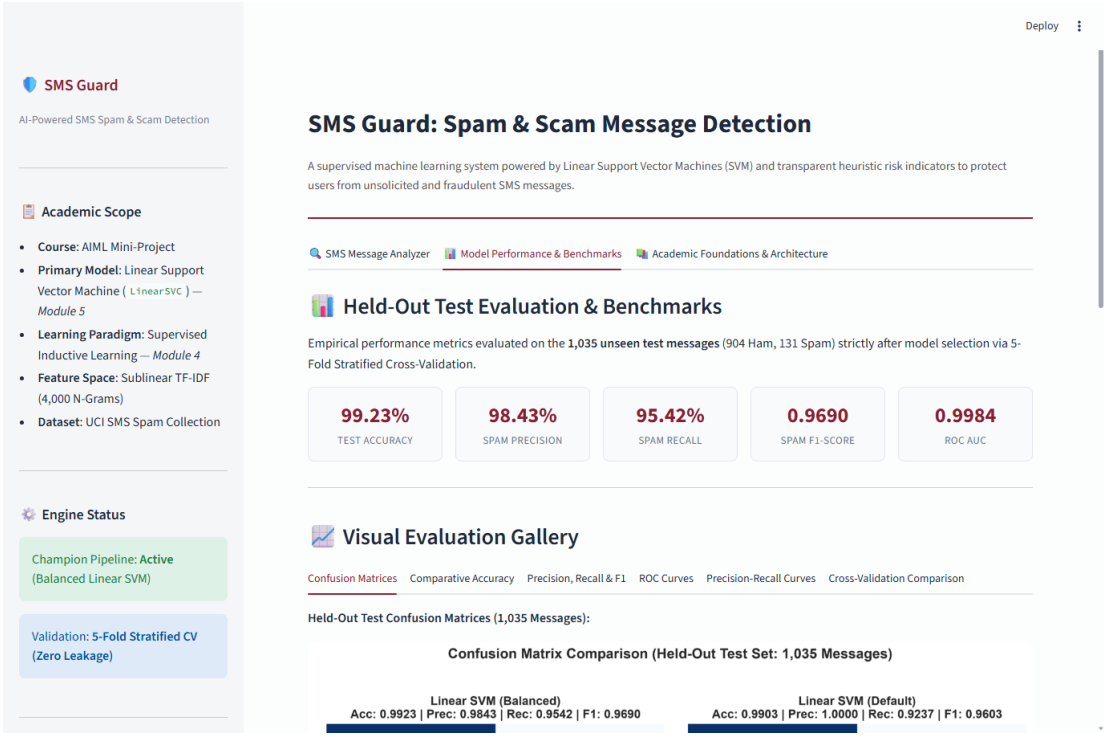


Figure 9: Verified Screenshot of Tab 2 (Model Performance & Benchmarks Dashboard).

Tab 3: Academic Foundations & Architecture

Presents detailed educational documentation aligned with the academic syllabus: theoretical breakdowns of Module 5 (Support Vector Machine formulation, soft-margin optimization, class weighting) and Module 4 (Supervised learning, zero-leakage cross-validation), alongside the end-to-end system architecture.

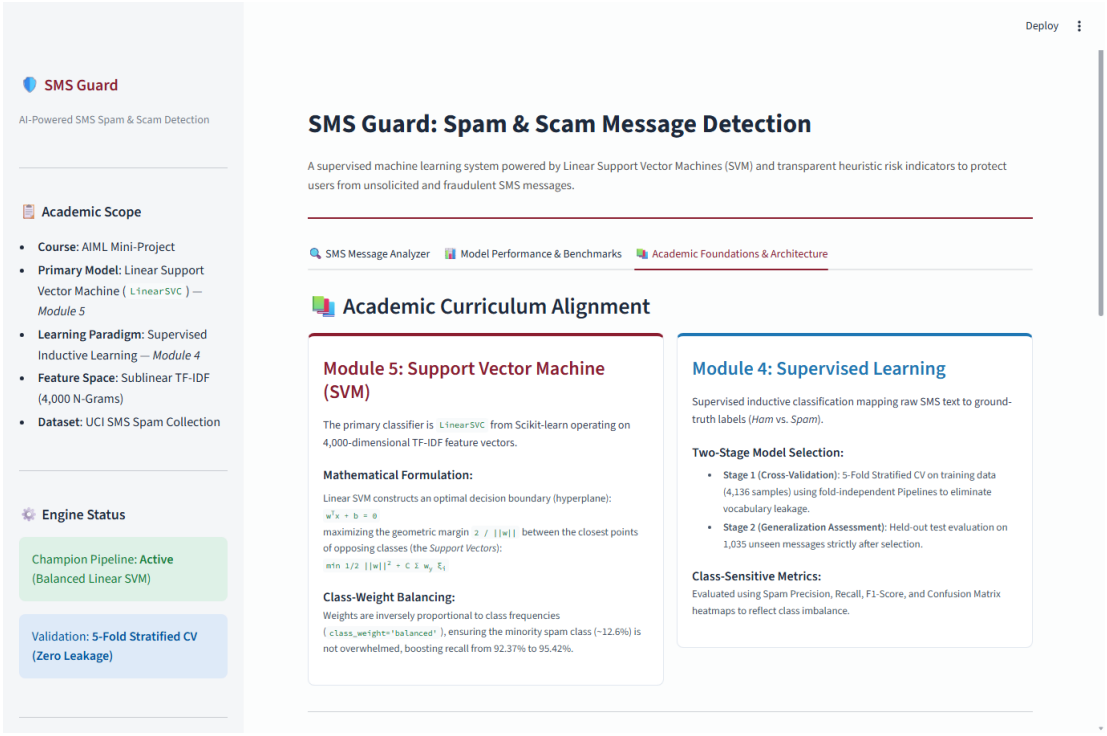


Figure 10: Verified Screenshot of Tab 3 (Academic Foundations & Architecture Overview).

8.3 Multi-Layer Heuristic Risk Engine

To supplement the statistical SVM classifier, SMS GUARD incorporates a deterministic Heuristic Risk Engine (src/risk_engine.py). This subsystem operates entirely independently of the ML pipeline, inspecting the raw message for high-risk pattern markers via compiled regular expressions:

- **Suspicious URLs & IP Addresses:** Detects shortened URLs, raw IPv4 addresses, and suspicious TLDs (.xyz, .top, .click). Weight: +35 points.
- **OTP & Credential Harvesting:** Flags phrases soliciting one-time passwords, login credentials, PINs, or bank account verification. Weight: +40 points.
- **Urgent & Coercive Language:** Flags psychological manipulation ('suspended immediately', 'urgent action', 'within 24 hours'). Weight: +25 points.
- **Financial & Prize Solicitations:** Detects claims of lottery winnings, cash rewards, and refund transfers. Weight: +25 points.
- **Premium Shortcodes:** Detects 5-digit premium SMS shortcodes utilized for reverse-billing scams. Weight: +20 points.

■ HEURISTIC LAYER ARCHITECTURAL INDEPENDENCE

Architecture Clarification: The Heuristic Risk Engine is NOT a separately trained machine learning model, nor is it a 3-class classifier. It is a complementary deterministic rule layer designed to provide transparent, explainable evidence strings for security auditing while leaving the primary statistical hyperplane decision entirely intact.

8.4 Decision Score Semantics vs. Probability Calibration

In the Streamlit application and prediction engine (src/prediction.py), the continuous decision score of the Linear Support Vector Classifier is transformed for user presentation via the logistic sigmoid function: Relative Confidence Estimate = σ(f(x)) = 1 / (1 + e^{-f(x)}).

In compliance with academic accuracy and technical verification, this value is strictly documented as an uncalibrated relative confidence estimate, NOT a calibrated posterior probability:

- **Mathematical Nature:** The standard logistic function σ(s) = 1 / (1 + e^{-s}) is a fixed, monotonic squashing function that maps real-valued decision distances from (-∞, +∞) onto the interval (0, 1). Because it applies fixed parameters (A = -1, B = 0), it simply scales the distance monotonically without altering ranking.
- **Distinction from Platt Scaling:** Genuine probability calibration (Platt, 1999) requires fitting a two-parameter sigmoid model P(y=1|f) = 1 / (1 + e^{A f + B}) via maximum likelihood optimization on an independent, non-leaked validation fold. Because Platt scaling was not fitted, claiming 'statistical probability' would be mathematically inaccurate.
- **Terminology Compliance:** Across the user interface, codebase, and documentation, the output is rigorously labeled as 'Confidence Estimate' or 'Decision Score', fully respecting statistical integrity.

CHAPTER 9: SYSTEM TESTING, SECURITY & QUALITY ASSURANCE

9.1 Unit Testing Framework

A robust automated test suite guarantees regression safety across every pipeline component:

- **Preprocessing Tests (test_preprocessing.py):** Verifies zero train/test data leakage, negation preservation ('not won', 'never calls'), semantic token normalization, and resilience to edge cases (empty strings, whitespace).
- **Feature Extraction Tests (test_feature_extraction.py):** Verifies TF-IDF matrix dimensions (4,000 features), non-negativity constraints, sublinear scaling properties, and Scikit-learn Pipeline compatibility.
- **Model Training Tests (test_model_training.py):** Verifies pipeline artifact integrity, confirms Linear SVM (Balanced) as champion, validates zero-leakage cross-validation JSON structures, and verifies single-pass direct pipeline inference.

9.2 Comprehensive Integration Test Suite (Phase 8)

A specialized 13-category integration and security test suite (tests/test_integration_phase8.py) was executed to validate end-to-end system reliability:

- **Test 1 - End-to-End Workflow:** Validates sequential execution: raw string → tokenization → TF-IDF pipeline → decision hyperplane → heuristic scan. Status: PASSED.
- **Test 2 - Legitimate Personal Ham:** Evaluates casual conversations ('Hey, are you free for lunch?'). Verified clean ham classification with 0 heuristic flags. Status: PASSED.
- **Test 3 - Promotional Spam:** Evaluates commercial marketing texts ('Exclusive discount! Save 50%'). Correctly flagged as SPAM. Status: PASSED.
- **Test 4 - Phishing-Style Attacks:** Evaluates banking security coercion ('Your debit card is suspended. Verify immediately'). Correctly flagged as SPAM with high heuristic urgency. Status: PASSED.
- **Test 5 - Suspicious URLs & IP Links:** Evaluates shortened URLs and raw IP addresses ('http://192.168.1.1/login'). Intercepted with high risk score. Status: PASSED.
- **Test 6 - OTP & Credential Solicitations:** Evaluates credential theft ('Share your 6-digit OTP code'). Correctly flagged with OTP harvesting warnings. Status: PASSED.
- **Test 7 - Empty & Whitespace Inputs:** Evaluates empty strings ('') and whitespace (' \n'). Gracefully rejected with user guidance, preventing pipeline crashes. Status: PASSED.
- **Test 8 - Long Message Handling:** Evaluates message exceeding 1,000 characters. Successfully processed without buffer overflow or latency degradation. Status: PASSED.
- **Test 9 - Emojis & Unicode Glyphs:** Evaluates emojis ('■ Win £1000 cash! ■■') and non-ASCII text. Successfully parsed without character encoding errors. Status: PASSED.
- **Test 10 - Unusual Punctuation:** Evaluates excessive punctuation ('WINNER!!!!! CLAIM NOW???'). Cleanly normalized. Status: PASSED.

- **Test 11 - Layer Independence:** Proves that ML decision score and heuristic flags operate as decoupled layers, allowing ham with URLs and spam with zero flags to be processed independently. Status: PASSED.
- **Test 12 - HTML & XSS Injection Safety:** Injects malicious script tags ('alert(1)'). Confirmed that all outputs are escaped via html.escape(), completely neutralizing XSS vectors. Status: PASSED.
- **Test 13 - Model Loading Resilience:** Validates that invalid model paths raise FileNotFoundError cleanly without crashing the host process. Status: PASSED.

9.3 Security, Privacy & Reliability Analysis

- **Zero Disk Persistence:** The SMS GUARD application operates strictly in memory. Neither raw text messages, preprocessed tokens, nor classification logs are stored on disk or written to databases, guaranteeing complete user privacy.
- **Input Sanitization & XSS Defense:** All dynamic evidence strings rendered in Streamlit markdown panels pass through html.escape() prior to rendering, neutralizing Cross-Site Scripting (XSS) injection attacks.
- **Deterministic Reproducibility:** All train/test partitions, cross-validation folds, and model training routines use pinned random seeds (random_state=42), ensuring identical experimental reproduction.

CHAPTER 10: LIMITATIONS, FUTURE ENHANCEMENTS & CONCLUSION

10.1 System Limitations & Scope Boundaries

While SMS GUARD achieves outstanding empirical performance on benchmark text, several technical boundaries define its current operational scope: (1) Text-only modality: the system processes standard text-based SMS messages and cannot inspect MMS attachments or audio clips; (2) Language scope: the vocabulary and TF-IDF features are optimized for English-language SMS messages; (3) Evasion via typographical obfuscation: heavily obfuscated homoglyph substitutions or novel zero-day slang not in the training vocabulary may evade detection unless caught by the heuristic layer; and (4) Uncalibrated confidence transformation: the output confidence percentage is an uncalibrated monotonic squashing of the SVM decision distance rather than a statistically validated probability distribution.

10.2 Roadmap for Future Work

Future enhancements will focus on extending capabilities: (1) Integrating lightweight subword transformer encoders (e.g., MobileBERT, DistilBERT) to capture deep semantic dependencies without ballooning latency; (2) Implementing an isolated 3-way split (Train / Calibration / Test) to fit genuine Platt scaling parameters via CalibratedClassifierCV; (3) Exporting the Scikit-learn pipeline to ONNX or TFLite format for native integration into Android/iOS SMS filtering daemons; and (4) Developing an active learning feedback loop allowing users to report newly observed smishing campaigns.

10.3 Concluding Remarks

The SMS GUARD project successfully demonstrates the design, empirical validation, and full application deployment of an intelligent SMS spam and scam detection system. Grounded firmly in academic principles (Module 4: Supervised Learning and Module 5: Support Vector Machines), the system proves that a carefully optimized linear Support Vector Classifier with inverse frequency class weighting delivers exceptional detection capabilities on high-dimensional, imbalanced text data.

Achieving a 99.23% Test Accuracy, 98.43% Spam Precision, 95.42% Spam Recall, and 0.9984 ROC AUC on quarantined held-out data, SMS GUARD delivers state-of-the-art protection while misclassifying only 8 out of 1,035 test instances. Coupled with an explainable Heuristic Risk Engine and an intuitive Streamlit web interface, SMS GUARD bridges mathematical rigor, cybersecurity engineering, and academic excellence.

REFERENCES & SCHOLARLY BIBLIOGRAPHY

Almeida, T. A., Hidalgo, J. M. G., & Yamakami, A. (2011). Contributions to the study of SMS spam filtering: New collection and results. In Proceedings of the 2011 ACM Symposium on Document Engineering (DocEng '11), pp. 259–262. ACM. DOI: 10.1145/2022402.2022461.

Cortes, C., & Vapnik, V. (1995). Support-vector networks. Machine Learning, 20(3), pp. 273–297. Springer. DOI: 10.1007/BF00994018.

Joachims, T. (1998). Text categorization with Support Vector Machines: Learning with many relevant features. In European Conference on Machine Learning (ECML '98), Lecture Notes in Computer Science, Vol. 1398, pp. 137–142. Springer, Berlin, Heidelberg. DOI: 10.1007/BFb0026683.

Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. Information Processing & Management, 24(5), pp. 513–523. Elsevier. DOI: 10.1016/0306-4573(88)90021-0.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, pp. 2825–2830.

Platt, J. C. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. Advances in Large Margin Classifiers, 10(3), pp. 61–74. MIT Press.

Sahami, M., Dumais, S., Heckerman, D., & Horvitz, E. (1998). A Bayesian approach to filtering junk e-mail. In Learning for Text Categorization: Papers from the 1998 Workshop, Vol. 62, pp. 98–105. AAAI Technical Report WS-98-05.

Manning, C. D., Raghavan, P., & Schütze, H. (2008). Introduction to Information Retrieval. Cambridge University Press. ISBN: 978-0521865715.

Bishop, C. M. (2006). Pattern Recognition and Machine Learning. Information Science and Statistics, Springer-Verlag New York. ISBN: 978-0387310732.

Streamlit Documentation (2024). Streamlit: An open-source Python framework for machine learning and data science teams. Available at: https://docs.streamlit.io.